Run Length Encoding

AlgoExpert · Easy · Strings

The problem

- Encode runs as count+char, with runs longer than 9 split ("AAAAAAAAAAAAA" → "9A3A").

Intuition

- Count runs of equal characters, but cap each run at nine so the count is always a single digit.

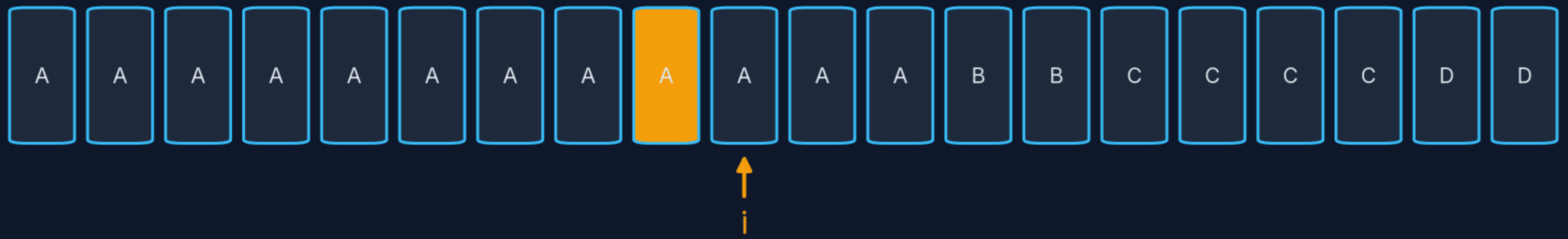
Brute force vs optimal

- Naive: string concatenation in a loop → $O(n^2)$
- Optimal: StringBuilder, single pass → $O(n)$
- Cap runs at 9 so counts stay one digit

Approach

- 1. Scan with a running count
- 2. Flush when the character changes or the count hits nine
- 3. Write count then character

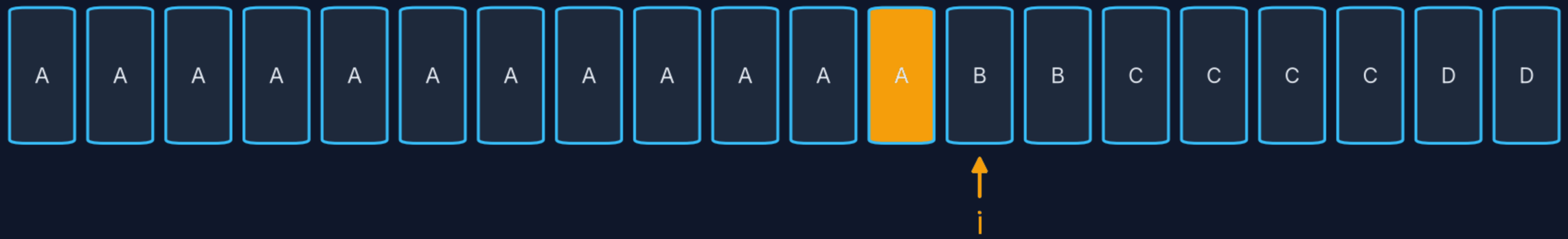
Worked example




```
flush 9A  (cap at 9)
output = "9A"
```

Flush run → 9A

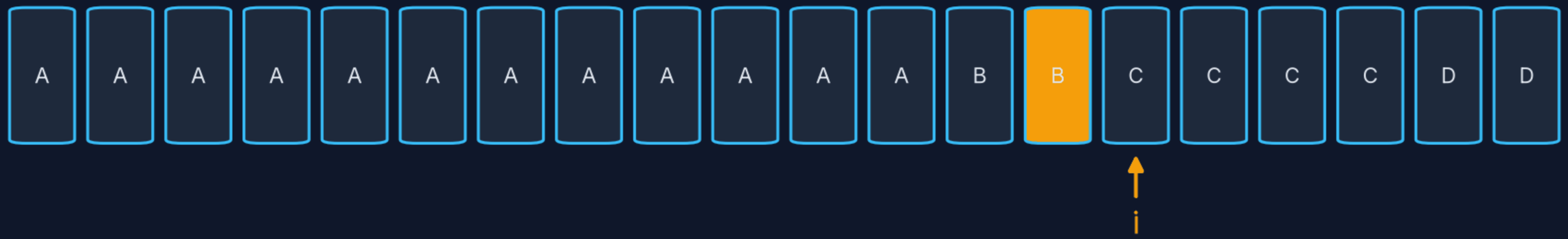
Worked example




```
flush 3A
output = "9A3A"
```

Flush run → 3A

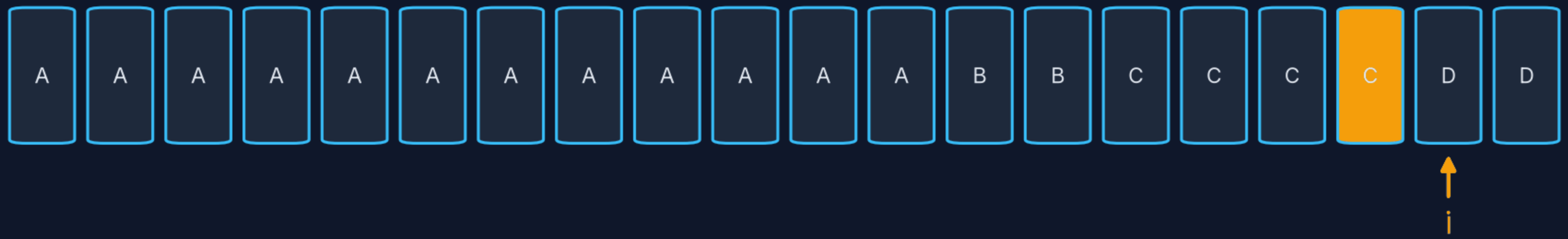
Worked example




```
flush 2B
output = "9A3A2B"
```

Flush run → 2B

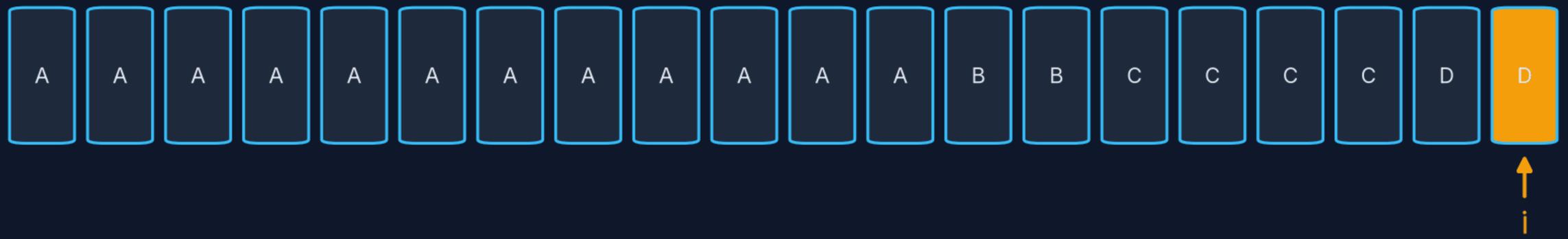
Worked example




```
flush 4C
output = "9A3A2B4C"
```

Flush run → 4C

Worked example




```
flush 2D
output = "9A3A2B4C2D"
```

Flush run → 2D

Java solution

```java
public class RunLengthEncoding {
    public static String runLengthEncoding(String s) {
        StringBuilder sb = new StringBuilder();
        int run = 1;
        for (int i = 1; i <= s.length(); i++) {
            if (i == s.length() || s.charAt(i) != s.charAt(i - 1) || run == 9) {
                sb.append(run).append(s.charAt(i - 1));
                run = 1;
            } else run++;
        }
        return sb.toString();
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Runs longer than 9 split (12 A's → 9A3A)
- Flush the final run after the loop
- Digits in the input

Interview tips & follow-ups

- Explain why the cap avoids ambiguity
- Follow-up: write the decoder

Recap

- Count runs of equal characters, but cap each run at nine so the count is always a single digit.
- Complexity: O(n)